

1- If you could use a clock with jitter only or skew only, which would you choose and why?

the Skew is a deterministic Spatial phenomenon while jitter is a Random temporal effect, which is very Difficult to Be Controlled.

In addition, the Skew may be used to Solve Setup/Hold violations depending on whether the Skew is +ve or -ve.

So, it would be better to use a clock with Skew.

2- Write an expression for a clock period suffering from jitter and for the hold-time suffering from negative skew

1) clock period suffering from jitter
assuming the worst case where the 1st edge jitter is +ve, while the 2nd edge jitter is -ve

$$T - 2T_j > t_{cq} + t_{pd} + t_{su}$$

2) Hold time suffering from -ve skew

$$t_{\text{hold}} - t_{\text{skew}} < t_{cq} + t_{pd} \rightarrow \text{More Relaxed Hold time Requirement}$$

3- Write an expression for hold-time under negative jitter and hold-time under positive jitter

Jitter doesn't have a direct impact on hold-time Requirements as if jitter moves the active edge, it moves it for all of the Registers

Simultaneously

advantage:-

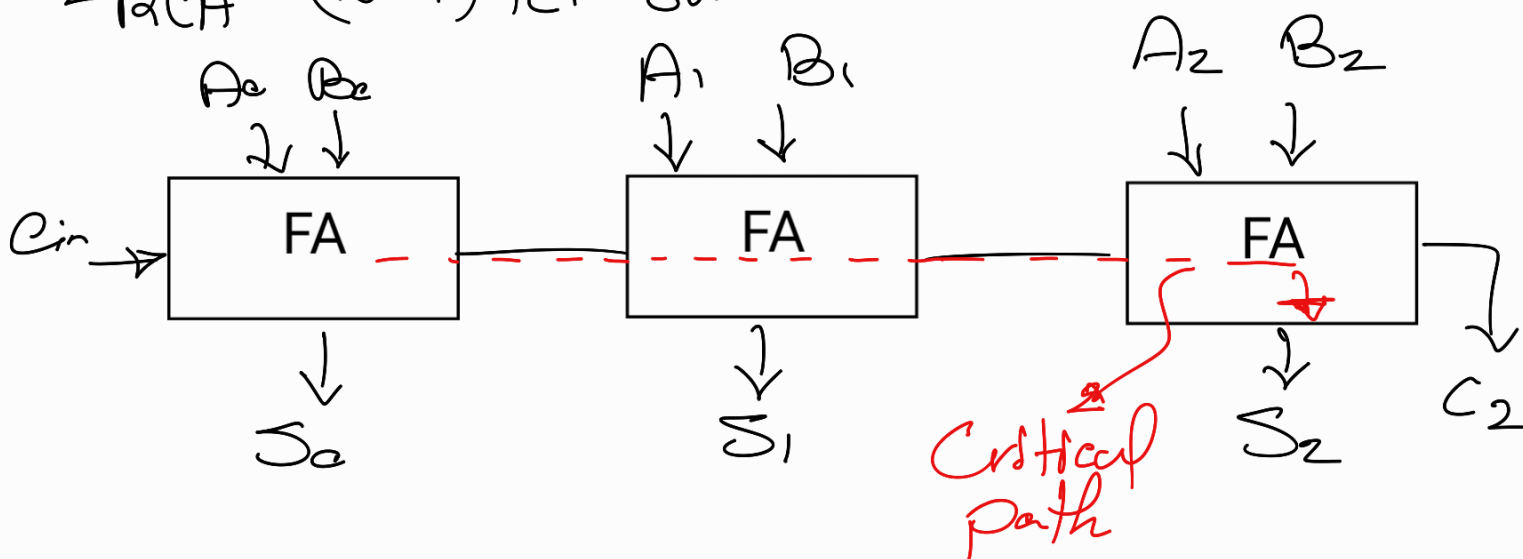
- 1) its area and power Budget is better than that of the grid
- 2) Minimizes the Relative delay on the same level

Disadvantage:-

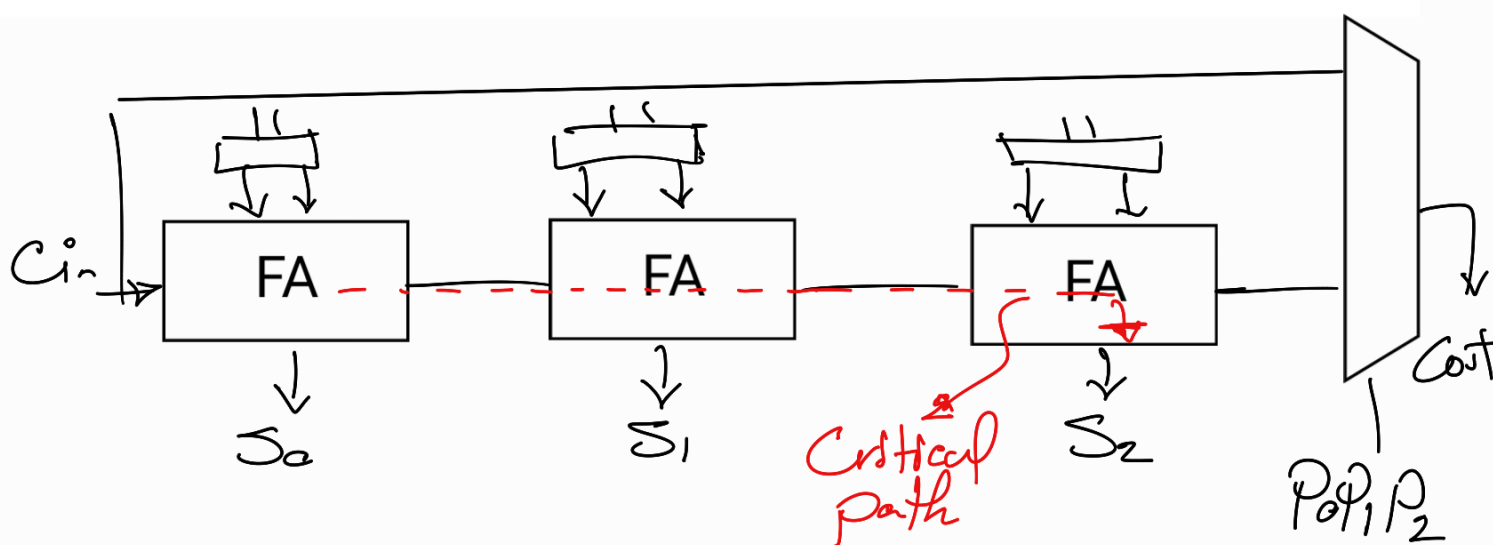
- * Not Design independent as it depends on the final location of the cells in the layout. Thus, if any change is made to the design, the Tree has to be Modified.

5-Write an expression for the delay of an N-bit ripple carry adder (define all variables you use if they differ from those in lecture)

$$t_{RCA} = (N-1)t_c + t_{sum}$$



6-Write an expression for the delay of an N-bit carry bypass adder (define all variables you use)



$$t_{Bypass} = t_{gp} + (N-1)t_c + t_{sum}$$

adder

7-At what total number of bits N does a carry bypass adder become more efficient than a ripple carry adder

* Consider p cascaded stages of the carry Bypass adder, No. of bits / Stage = k

$$\begin{aligned} \therefore t_{\text{Bypass adder}} &= kt_c + t_{\text{mux}} + (p-2)t_{\text{mux}} + (k-1)t_c + t_{\text{sum}} \\ &= kt_c + \left(\frac{N}{k} - 1\right)t_{\text{mux}} + (k-1)t_c + t_{\text{sum}} \end{aligned}$$

$$\therefore t_{\text{RCA}} = (N-1)t_c + t_{\text{sum}}$$

For the Bypass adder to be more efficient we need $t_{\text{Bypass}} < t_{\text{RCA}}$

$$kt_c + \left(\frac{N}{k} - 1\right)t_{\text{mux}} + (k-1)t_c + t_{\text{sum}} < (N-1)t_c + t_{\text{sum}}$$

$$N\left(t_c - \frac{t_{\text{mux}}}{k}\right) > t_c(2k-1) - t_{\text{mux}}$$

Assume $t_{\text{mux}} = t_c$ for simplicity

$$\therefore N\left(\frac{k-1}{k}\right) > 2k-1$$

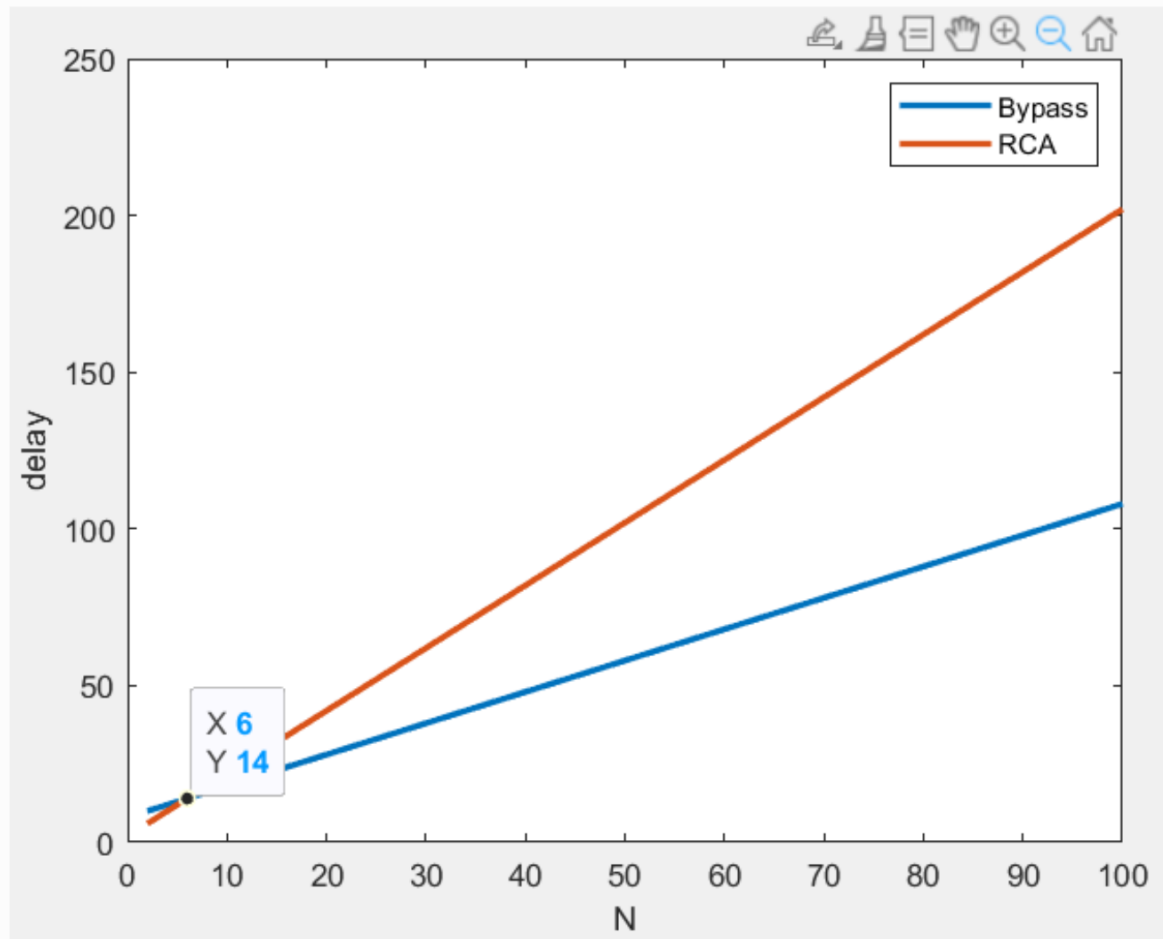
$$N > \frac{k(2k-1)}{k-1}$$

* For example at $k=2 \rightarrow N > 6$ assume $\boxed{N=8}$

$$\therefore t_{\text{RCA}} \rightarrow 7t_c + t_{\text{sum}}$$

$$t_{\text{Bypass}} \rightarrow 2t_c + 3t_{\text{mux}} + t_c + t_{\text{sum}} \approx 6t_c + t_{\text{sum}} \quad \text{for } t_c = t_{\text{mux}}$$

→ Matlab Result



* additional Question

Same as 7 but Replace Carry Bypass adder with the carry select adder

Sol.

↳ Carry Select adder?

* assume the no. of Bits for stage $i = K + i - 1$

$$\begin{aligned} \circ \circ N &= K + (K+1) + (K+2) + (K+3) - \dots + (K+p-1) \\ &= \frac{\rho(K + K + \rho - 1)}{2} \approx \frac{\rho^2}{2} \rightarrow \rho = \sqrt{2N} \end{aligned}$$

$$\circ \circ t_{\text{carry sel.}} = K t_c + (\sqrt{2N} - 1) t_{\text{mux}} + (K + \rho - 1 - 1) t_c + t_s$$

assume $t_{\text{mux}} = t_c$

$$\begin{aligned} \circ \circ t_{\text{carry sel.}} &= t_c (K + \sqrt{2N} - 1 + K + \rho - 2) + t_s \\ &= t_c (2K + 2\sqrt{2N} - 3) + t_s \end{aligned}$$

for the Carry Sel adder to be more eff.

$$t_{\text{carry sel.}} < t_{\text{RCFA}}$$

$$t_c (2K + 2\sqrt{2N} - 3) + t_s < (N - 1) t_c + t_s$$

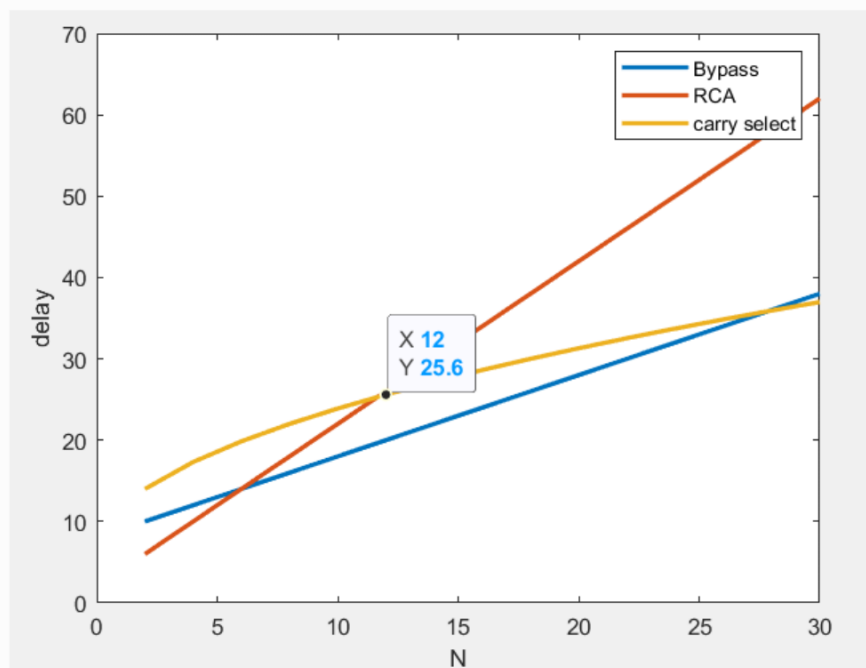
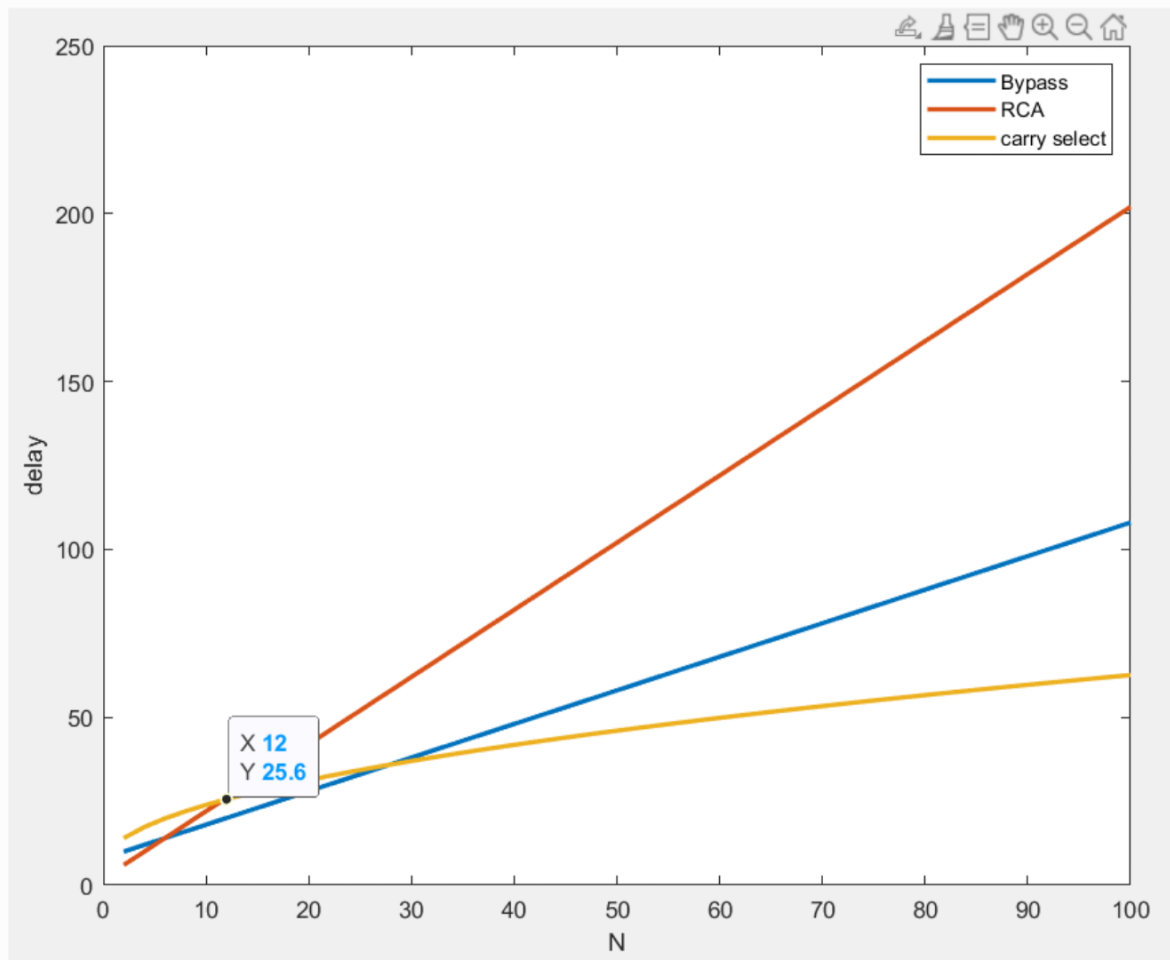
$$N > 2K + 2\sqrt{2N} - 2$$

$$N - 2\sqrt{2} \times \sqrt{N} + 2 - 2K > 0$$

$$\underline{\text{Sol.}} \rightarrow \sqrt{N} = \frac{2\sqrt{2}(1 + \sqrt{K})}{2} = \sqrt{2}(1 + \sqrt{K})$$

$\circ \circ$ at $N > 2(1 + \sqrt{K})^2$, the carry sel. is more eff.

→ Matlab Result



8-Does raising the number of bits per block in the carry bypass adder reduce or increase the result in part 7, explain why this happens briefly

* Raising k increases the Required total No. of Bits to make the Carry Bypass adder more efficient

* Explanation:-

- 1) Considering the worst case Scenario at which $k=N$, the Carry Bypass adder Critical path will be similar to that of the RCA.
- 2) The Rate at which t_{pd} increases in the Case of the Carry Bypass adder is $\frac{t_{mux}}{k}$
Thus, we have to decrease this Rate in order to make the Carry Bypass adder more efficient compared to the RCA.

1) Empty Flag case

- * the FIFO is Empty when the Read pointer increments and catches the write pointer pointing to the first empty location. Thus, this indicates the FIFO is empty as the Read pointer is pointing to an empty location.
- * Due to the one cycle uncertainty, the Synchronized version of the write pointer being passed to the Rx domain may be a lagged version of the original one. Reading a lagged version of the write pointer will result in a fake empty flag assertion but this isn't a failure. However, it may be considered as overhead.

2) Full Flag case

- * the FIFO is full When the write pointer increments and catches the Read pointer pointing to the first occupied location. Thus, this indicates the FIFO is full as the write pointer is pointing to an occupied location.
- * Due to the one cycle uncertainty, the Synchronized version of the read pointer being passed to the Tx domain may be a lagged version of the original one. Reading a lagged version of the read pointer will result in a fake Full Flag assertion but this isn't a failure. However, it may be considered as overhead.

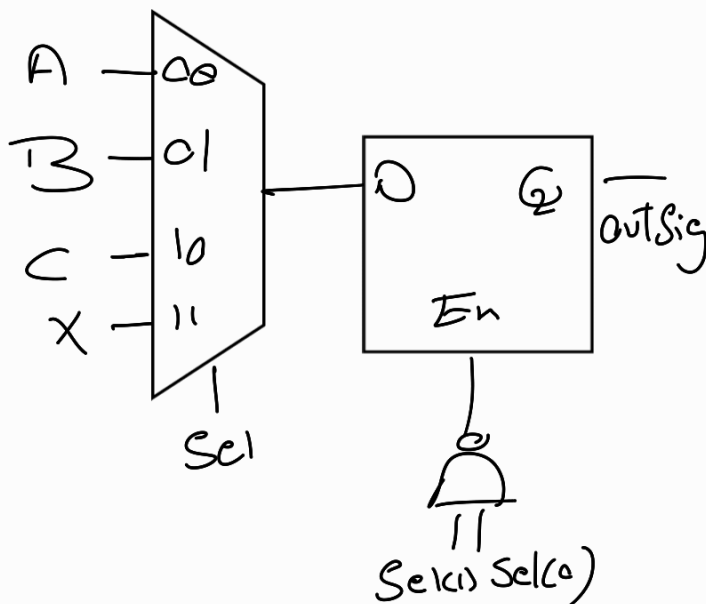
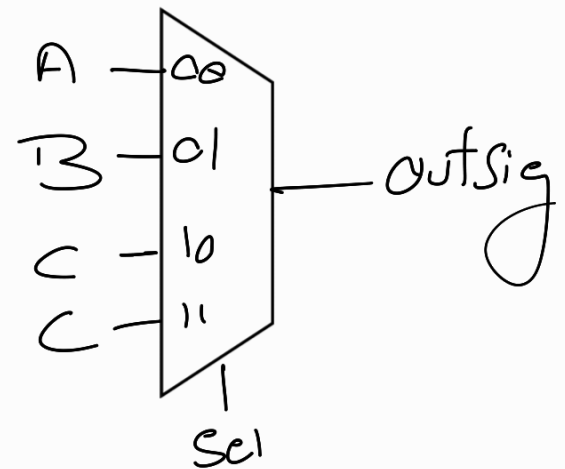
10- Draw an equivalent circuit for these two snippets of code

Circuit 1	Circuit 2
<pre> Process(sel, A, B, C) Begin If (sel="00") then Outsig <= A; Elsif (sel="01") then Outsig <= B; Elsif (sel="10") then Outsig <= C; End if; End process; </pre>	<pre> Process(sel, A, B, C) Begin If (sel="00") then Outsig <= A; Elsif (sel="01") then Outsig <= B; Else Outsig <= C; End if; End process; </pre>

* there is no Else Branch in this process. So, the Synthesis tool infers a latch as the process doesn't cover all the possible values for "sel"

* the "sel" cases are mutually exclusive ones. Thus, the tool infers a parallel 4x1 Mux

* No latch as the else Branch is included



in the two-port RAM, the Memory array has to be assigned using two separate processes. Thus, declaring the array as a signal results in a compiler or Synthesizer error. While, in the single port RAM, the array is accessed using a single process. So, it may be declared as a signal.

12-Technology scaling occurs by a ratio S . However, global wire length scaling is constant (i.e. scales by 1). To control wire resistance, we also scale wire height (thickness) by 1. All other dimensions including wire width and separation scale at the same rate as gate dimensions (S). Find an expression for the percentage increase in interwire capacitance between two adjacent parallel wires of the same length.

* the problem states that:

1) the global wire length doesn't scale down

2) t_w is constant

* the inter-wire capacitance is given by:-

$$C_{\text{inter wire}} = \frac{\epsilon \times L t_w}{L_{\text{sep}}}$$

∴ $L, t_w \rightarrow \text{constant}$, $L_{\text{sep}} \propto \frac{1}{S}$

∴ $C_{\text{inter wire}} \propto S$

$$\begin{aligned} \text{* Percentage inc.} &= \frac{S \times C_{\text{inter wire}} - C_{\text{inter wire}}}{C_{\text{inter wire}}} \times 100 \\ &= (S-1) \times 100 \end{aligned}$$

13-If voltage drops by a ratio U when scaling in question 12, find an expression for current scaling in the wire. Find an expression for current density scaling in the wire.

$$I_{\text{wire}} \propto \frac{V}{R_{\text{wire}}} \quad , \quad R_{\text{wire}} = \frac{L}{\sigma W t_w}$$

$$\circ \circ L, t_w \rightarrow \text{constant}$$

$$\circ \circ R_{\text{wire}} \propto \frac{1}{W}$$

$$\circ \circ I_{\text{wire}} \propto \frac{1}{W} \rightarrow \text{Current Scaling}$$

$$\ast \text{Current Density is given by: } I_{\text{wire}} \times \frac{1}{\text{Across Section}}$$

$$= I_{\text{wire}} \times \frac{1}{\underbrace{W t_w}_{\text{Constant}}}$$

$$\circ \circ \frac{1}{W} \propto S, \quad t_w \text{ is constant}$$

$$\circ \circ I_{\text{per unit Area}} \propto \frac{1}{W}$$

13-A local wire is allowed to scale in all dimensions (including length) at a rate similar to gate scaling (S). Find an expression for the percentage increase in wire resistance.

* the problem states that $L, w, tw \propto 1/S$

$$\therefore R_{wire} = \frac{L}{wA} \propto \frac{L}{wtw}$$

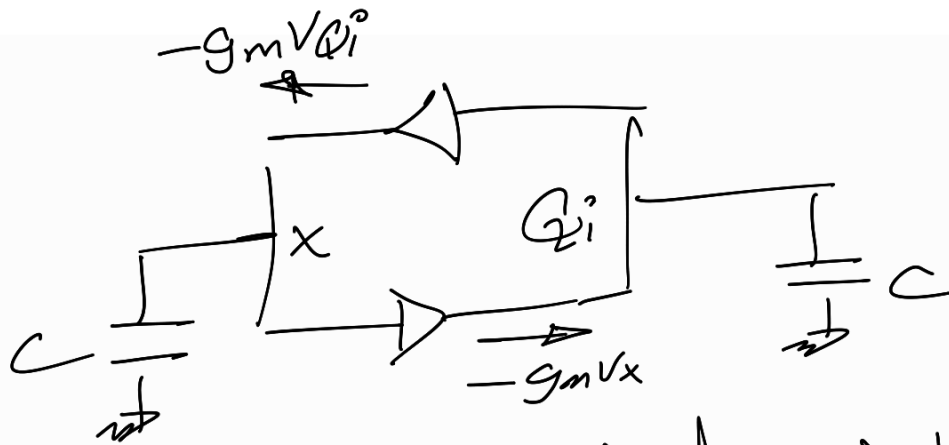
$$\therefore R_{wire} \propto S$$

$$\therefore \text{Percentage inc.} = \frac{S \cdot R_0 - R_0}{R_0} \times 100 = (S-1) \times 100$$

14- Show that the time to exit metastability is:

$$T_{exit} = \tau \ln(V_{DD}/V_d(0))$$

Start from the sketch of the two inverters as amplifiers



* assumptions: 1) the transconductance is the same (g_m)
2) the capacitive loading at the outputs of both inverters is equal

$$C \frac{dV_{Q1}}{dt} = -g_m V_x \quad (1)$$

$$C \frac{dV_x}{dt} = -g_m V_{Q1} \quad (2)$$

$$\text{Subtracting (1), (2)} \rightarrow C \frac{dV_d}{dt} = -g_m V_d$$

$$\text{the sol.} \rightarrow V_d = V_d(0) e^{-t/\tau}, \quad \tau = C/g_m$$

$$e^{-t/\tau} = \ln \left(\frac{V_d}{V_d(0)} \right)$$

for Metastability to End $\rightarrow V_d = V_{DD}$

$$T_{exit} = \tau \ln \left(\frac{V_{DD}}{V_d(0)} \right)$$

15- The following is a next state process for a state machine. It is written badly. Rewrite it and briefly point out the problem.

```
Process(reset, clk)
Begin
    If clk'event and clk='1' then
        If reset='1' then
            Next_state <= reset;
        Else
            Next_state <= current_state;
        End if;
    End if;
End process;
```

* This Code Snippet has 3 issues:-

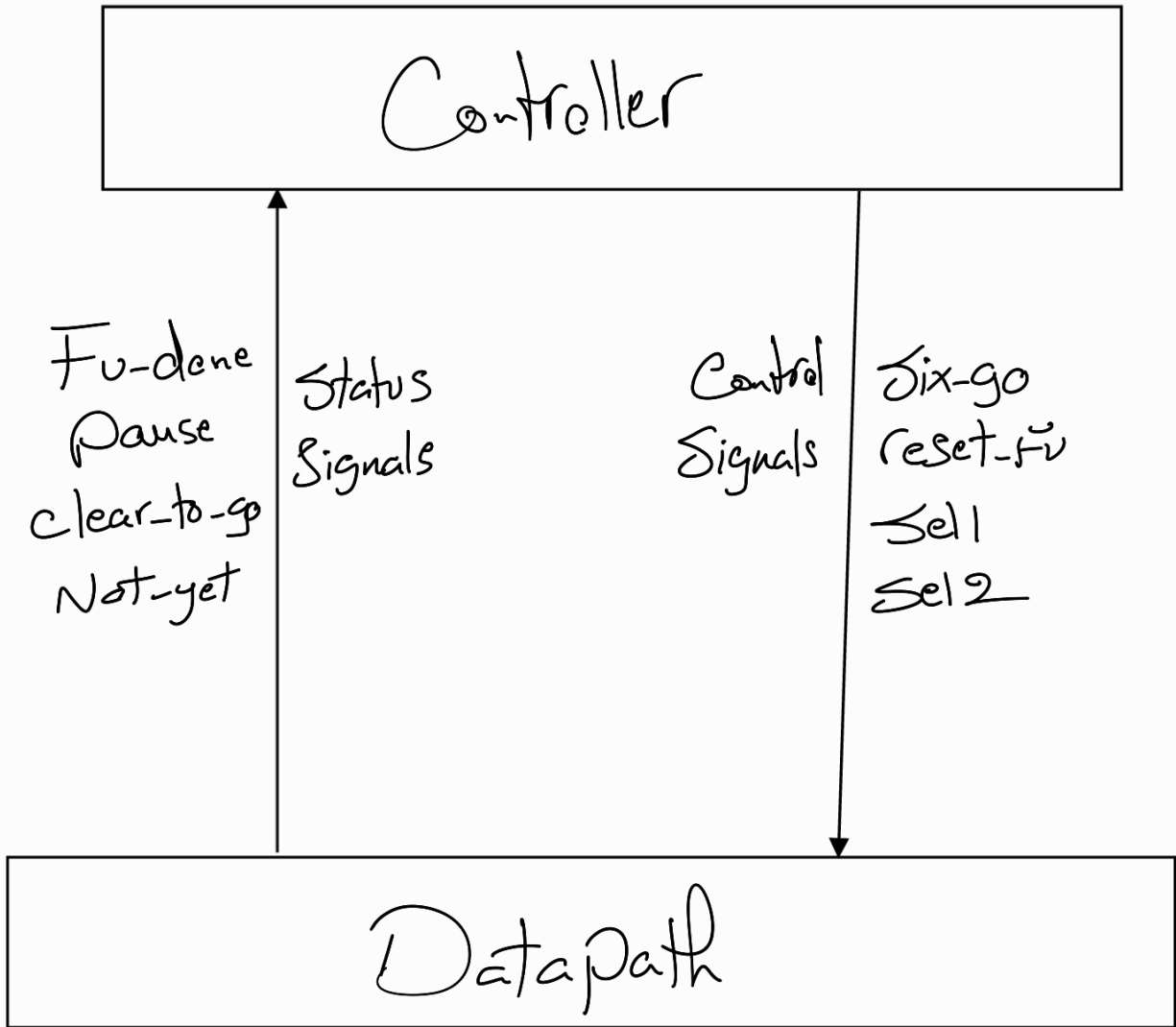
- 1) the Reset Signal is included in the Sensitivity list, so it is meant to be an asynchronous one
- 2) the Current State shall be loaded with the Next State and Not vice versa
- 3) at Reset, the Current State shall be the reset State which can't be named "reset" as there is a "reset" Signal

* the Code may be rewritten as follows

```
1  process( clk, reset )
2  begin
3      if reset = '1' then
4          current_state <= reset_state;
5      elsif clk'event and clk = '1' then
6          current_state <= next_state;
7      end if ;
8  end process ;
```

16-Sketch a block diagram showing control signals and status signals of the state machine described by these two processes. Neither process is anywhere near complete, but the code given is enough to give you the information.

Next state process	Output process
Process (FU_done, Pause, Clear_to_go, Not_yet, current_state) Begin End process;	When reset => Six_go <= '1'; reset_FU='1'; Sel1<='001'; Sel2<='002';



17- Draw the equivalent circuit of the following (nothing is case sensitive):

Process(clk, reset)

Begin

If reset = '1' then

A1p<=others=>'0'; b1p<=others=>'0'; c1p<=others=>'0'; d1p<=others=>'0'; e1p<=others=>'0';
f1p<=others=>'0';

A2p<=others=>'0'; b2p<=others=>'0'; c2p<=others=>'0'; d2p<=others=>'0'; e2p<=others=>'0';

A3p<=others=>'0'; b3p<=others=>'0'; c3p<=others=>'0'; d3p<=others=>'0';

A4p<=others=>'0'; b4p<=others=>'0';

A5p<=others=>'0';

Elsif clk'event and clk='1' then

A1p<=a1; b1p<=b1; c1p<=c1; d1p<=d1; e1p<=e1; f1p<=f1;

A2p<=a2; b2p<=b2; c2p<=c2; d2p<=d2; e2p<=e2;

A3p<=a3; b3p<=b3; c3p<=c3; d3p<=d3;

A4p<=a4; b4p<=b4;

A5p<=a5;

End if;

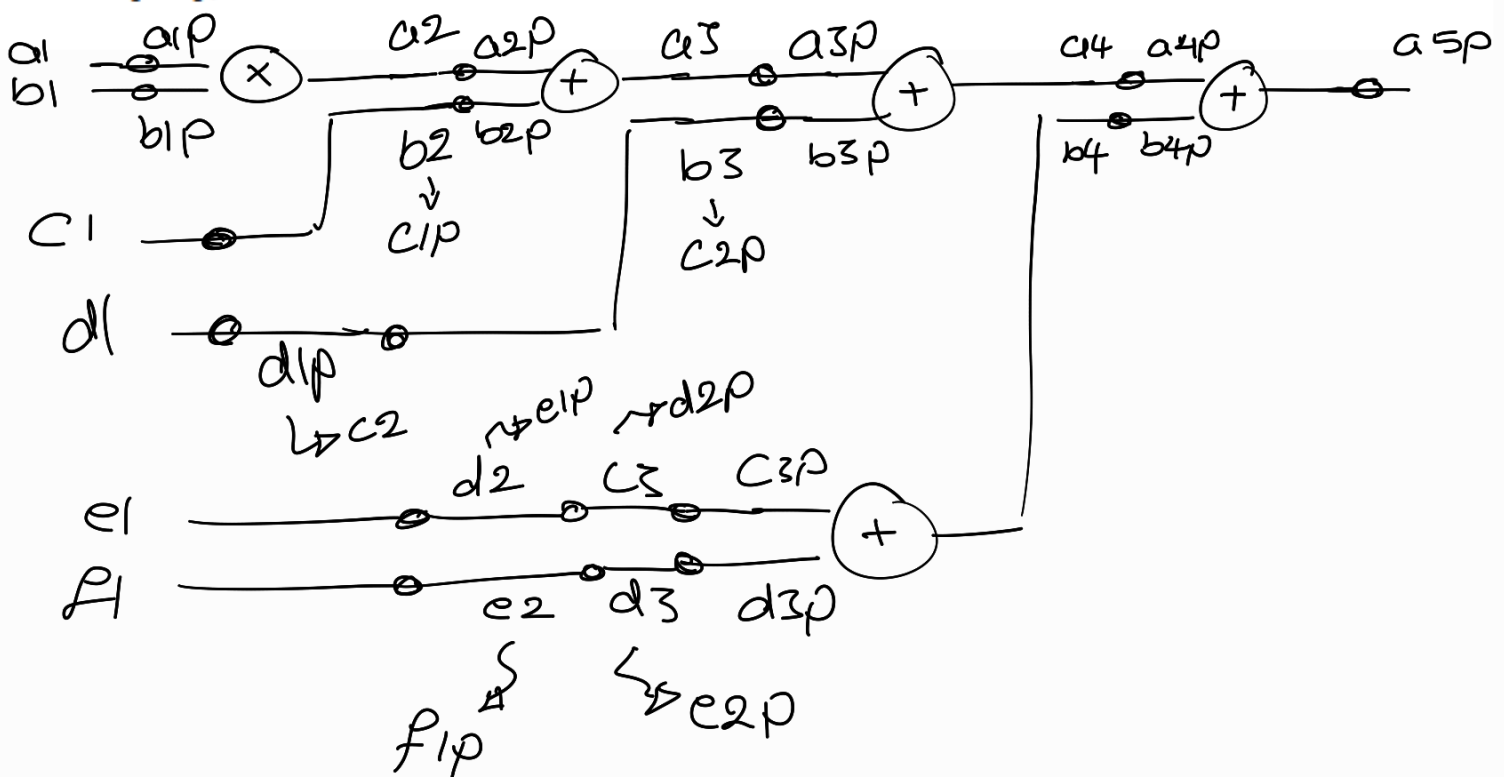
End process;

A2<=a1p*b1p; B2<=c1p; C2<=d1p; D2<=e1p; E2<=f1p;

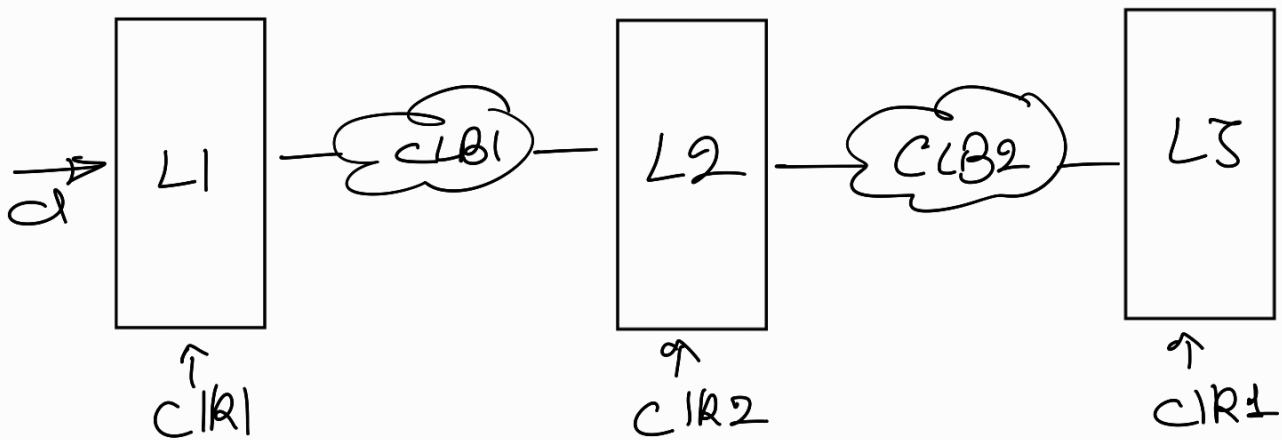
A3<=a2p+b2p; B3<=c2p; C3<=d2p; D3<=e2p;

A4<=a3p+b3p; B4<=c3p+d3p;

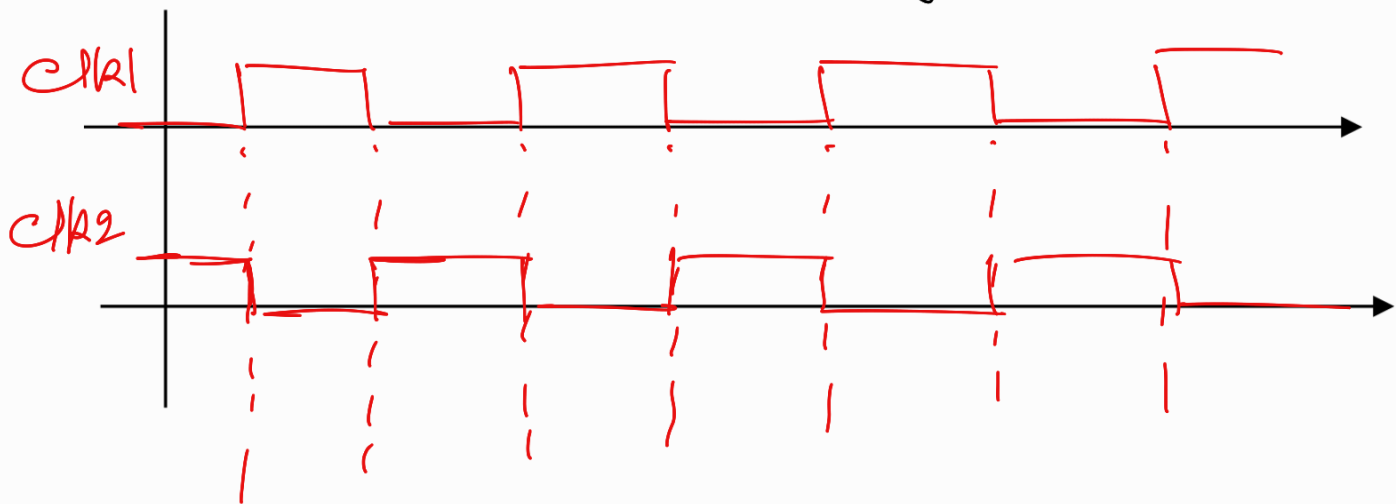
A5<=a4p+b4p;



* Considering the following latch loop



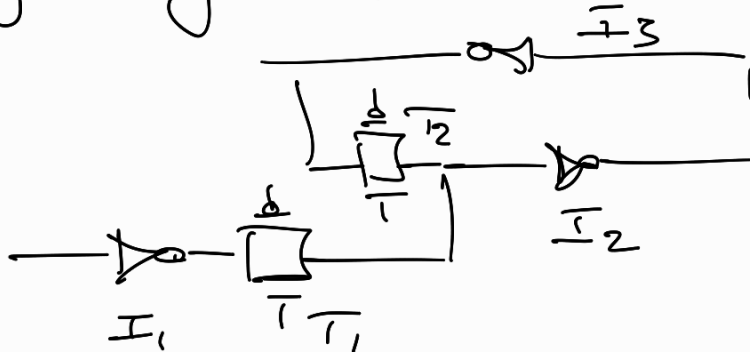
* CLK1, 2 are non-overlapping clocks



* Assuming No Slack Borrowing, the following Condition has to be Satisfied

$$\frac{T}{2} > t_{dq} + t_{pd}$$

* t_{dq} is given by: $t_{I_1} + t_{I_2} + t_{I_3}$



19-Write the expression for the condition that needs to be satisfied for total borrowing in every stage of a latch loop so that it is still functional

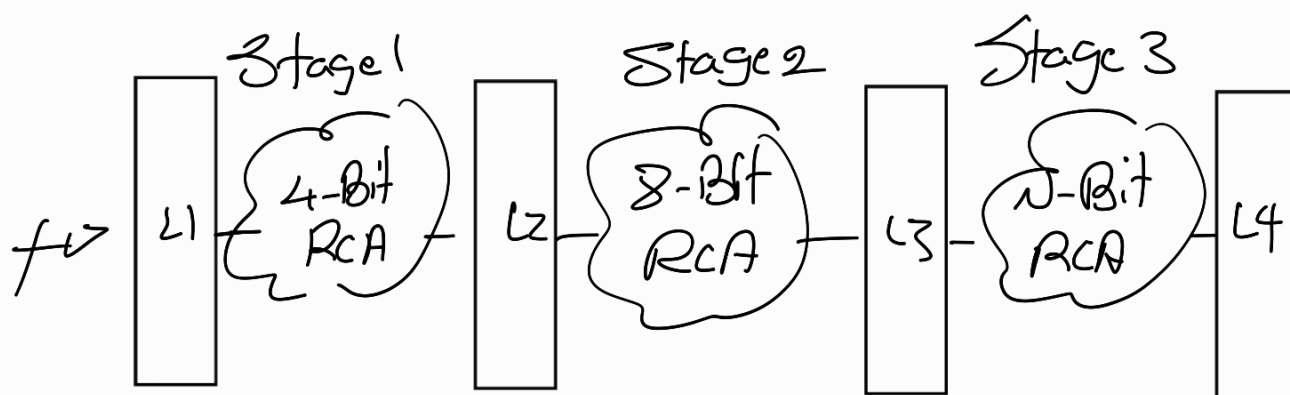
* the condition for total Borrowing

$$\sum_{2j} t_{\text{Borrow}j} < \frac{T}{2} - t_{SV}$$

$$\text{where } t_{\text{Borrow}j} = t_{dqj} + t_{pdj} - \frac{T}{2}$$

t_{ck}

20-For this question assume $t_{cq}=t_{su}=0$ for all latches. Also assume that for a FA $t_{sum}=5t_c$. A latch loop consists of three stages. The first is a 4-bit ripple carry adder. The second is an 8-bit ripple carry adder. The clock period T for this loop is $20t_c$. The third stage is an N -bit ripple carry adder. What is the maximum possible value for N that allows this latch loop to function properly?



∴ $\sum_{j=1}^n t_{Borrow_j}$ has to be less than $\frac{T}{2} - t_{su}$

$$\begin{aligned} \text{∴ } t_{\text{Stage 1}} &= 3t_c + \underbrace{t_s}_{5t_c} = 8t_c \rightarrow t_{\text{Borrow 1}} = t_{\text{Stage 1}} - \frac{T}{2} \\ &= 8t_c - 10t_c \\ &= -2t_c \end{aligned}$$

(دی میں فٹائر ہو گیا لیکن دی اول Stage کا کانت ہفٹرف بہت خلی صائر)

$$\begin{aligned} \text{∴ } t_{\text{Stage 2}} &= 7t_c + t_s = 12t_c \rightarrow t_{\text{Borrow 2}} = t_{\text{Stage 2}} - \frac{T}{2} \\ &= 12t_c - 10t_c \\ &= 2t_c \end{aligned}$$

$$\begin{aligned} \text{∴ } t_{\text{Stage 3}} &= (N-1)t_c + \underbrace{t_s}_{5t_c} \\ &= (N+4)t_c \rightarrow t_{\text{Borrow 3}} = (N+4)t_c - 10t_c \\ &= (N-6)t_c \end{aligned}$$

$$\text{∴ } 2t_c + (N-6)t_c < 10t_c$$

$$\underline{\underline{N < 14}}$$